

DIGITAL ELECTRONICS NOTES

Major Project Stage-1 Academic Course Notes
Autonomous Engineering Curriculum (JNTUH R18 / R22 Aligned)
Complete 130-Page E-Learning Reference Material

Course Syllabus (Units I - V)

UNIT -I:

Number System and Boolean Algebra :

Number Systems, Base Conversion Methods, Complements of Numbers, Codes- Binary Codes, Binary Coded Decimal Code and its Properties, Unit Distance Codes, Error Detecting and Correcting Codes.

Digital Logic Gates(AND,NAND,OR,NOR,EX-OR,EX-NOR), Properties of XOR Gates,

Universal Gates, Basic Theorems and Properties, Switching Functions, Canonical and Standard Form.

UNIT -II:

Minimization Techniques:

Introduction, The minimization with theorems, The Karnaugh Map Method, Three, Four and Five variable K- Maps, Prime and Essential Implications, Don't Care Map Entries, Using the Maps for Simplifying, Quine-McCluskey Method, Multilevel NAND/NOR realizations.

UNIT -III:

Combinational Circuits:

Design Procedure – Half Adder, Full Adder, Half Subtractor, Full Subtractor, Parallel Binary Adder, Parallel binary subtractor, Binary Multiplier, Multiplexers/DeMultiplexers, decoder, Encoder, Code Converters, Magnitude Comparator.

classification of sequential circuits, The binary cell, The S-R-Latch Flip-Flop The D-Latch Flip-Flop, The "Clocked T" Flip-Flop, The " Clocked J-K" Flip-Flop, Design of a Clocked Flip-Flop, Timing and Triggering Consideration.

UNIT -IV:

Sequential Circuits:

Introduction, Basic Architectural Distinctions between Combinational and Sequential circuits, Latches, Flip-Flops, SR, JK, D, T and Master slave, characteristic Tables and equations, Conversion from one type of Flip-Flop to another,

Counters - Design of Single Mode Counter, Ripple Counter, Ring Counter, Shift Register, Ring counter using Shift Register

UNIT -V:

Memory Devices:

Classification of memories – ROM : ROM organization, PROM, EPROM, EEPROM, RAM: RAM organization, Write operation, Read operation, Static RAM , Programmable Logic Devices: Programmable Logic Array(PLA), Programmable Array Logic, Implementation of

Combinational Logic circuits using ROM, PLA, PAL.

Text Books, Reference Books & Course Outcomes

TEXT BOOKS:

1. Digital Design- Morris Mano, PHI, 3rd Edition.
2. Switching Theory and Logic Design-A. Anand Kumar, PHI, 2nd Edition.
3. Switching and Finite Automata Theory- Zvi Kohavi & Niraj K. Jha, 3rd Edition, Cambridge.

REFERENCE BOOKS:

1. Introduction to Switching Theory and Logic Design – Frediac J. Hill, Gerald R. Peterson, 3rd Ed, John Wiley & Sons I
2. Digital Fundamentals – A Systems Approach – Thomas L. Floyd, Pearson, 2013.
3. Switching Theory and Logic Design – Bhanu Bhaskara – Tata McGraw Hill Publication, 2012
4. Fundamentals of Logic Design- Charles H. Roth, Cengage Learning, 5th, Edition, 2004.
5. Digital Logic Applications and Design- John M. Yarbrough, Thomson Publications, 2006.
6. Digital Logic and State Machine Design – Comer, 3rd, Oxford, 2013.

OUTCOMES:

Upon completion of the course, student should possess the following skills:

- Be able to manipulate numeric information in different forms
- Be able to manipulate simple Boolean expressions using the theorems and postulates of Boolean algebra and to minir
- Be able to design and analyze small combinational circuits and to use standard combinational functions to build large
- Be able to design and analyze small sequential circuits and to use standard sequential functions to build larger more c

Course Index

INDEX

S. No	Unit	Topic	Page no
1	I	NUMBERS SYSTEMS AND BOOLEAN ALGEBRA	01
2	II	MINIMIZATION TECHNIQUES	39
3	III	COMBINATIONAL CIRCUITS	60
4	IV	SEQUENTIAL CIRCUITS	89
5	V	MEMORY DEVICES	119

Total Document Length: 130 Pages

Prescribed Curriculum: Autonomous Engineering Colleges / JNTUH R18, R22

UNIT - 1: NUMBER SYSTEMS & BOOLEAN ALGEBRA

- Introduction about digital system
- Philosophy of number systems
- Complement representation of negative numbers
- Binary arithmetic
- Binary codes
- Error detecting & error correcting codes
- Hamming codes

INTRODUCTION ABOUT DIGITAL SYSTEM

A Digital system is an interconnection of digital modules and it is a system that manipulates discrete elements of information that is digital. Now a day's digital systems are used in wide variety of industrial and consumer products such as automated industrial machinery.

Characteristics of Digital systems:

- Digital systems manipulate discrete elements of information.
- Discrete elements are nothing but the digits such as 10 decimal digits or 26 letters of alphabets and so on.
- Digital systems use physical quantities called signals to represent discrete elements.
- In digital systems, the signals have two discrete values and are therefore said to be binary.
- A signal in digital system represents one binary digit called a bit. The bit has a value either 0 or 1.

Analog systems vs Digital systems:

Analog system process information that varies continuously i.e; they process time varying signals that can take on any values across the range of the signal.

Digital systems use digital circuits that can process digital signals which can take either 0 or 1 for binary system.

Advantages of Digital system over Analog system:**1. Ease of programmability:**

The digital systems can be used for different applications by simply changing the program without additional changes in hardware.

2. Reduction in cost of hardware:

The cost of hardware gets reduced by use of digital components and this has been possible due to advances in IC technology.

3. High speed:

Digital processing of data ensures high speed of operation which is possible due to advances in Digital Signal Processing.

4. High Reliability:

Digital systems are highly reliable one of the reasons for that is use of error correction codes.

5. Design is easy:

The design of digital systems which require use of Boolean algebra and other digital techniques is easier compared to analog design.

6. Result can be reproduced easily:

Since the output of digital systems unlike analog systems is independent of temperature, noise, humidity and other characteristics.

Disadvantages of Digital Systems:

- Use more energy than analog circuits to accomplish the same tasks, thus producing more heat as well.
- Digital circuits are often fragile, in that if a single piece of digital data is lost or misinterpreted the meaning of large blocks of data is lost.
- Digital computer manipulates discrete elements of information by means of a binary code.
- Quantization error during analog signal sampling.

NUMBER SYSTEM

Number system is a basis for counting various items. Modern computers communicate and operate with binary numbers which use only two digits, 0 and 1. For Ex: Let us consider decimal number 18. This number is represented in binary as 10010.

We observe that binary number system takes more digits to represent the decimal number. For large numbers we have to deal with a large number of digits.

- i) Octal number systems (base 8)
- ii) Hexa Decimal number system (base 16)
- iii) Binary Coded Decimal number (BCD) system

To define any number system we have to specify:

- **Base of the number system such as 2, 8, 10 or 16.**
- **The base decides the total number of digits available in that number system.**
- **First digit in the number system is always zero and last digit in the number system is always base-1.**

Binary number system:

The binary number has a radix of 2. As $r = 2$, only two digits are needed, and these are 0 and 1. In binary system weight is expressed as powers of 2. The left most bit, which has the greatest weight is called the Most Significant Bit (MSB). And the right most bit which has the least weight is called the Least Significant Bit (LSB).

For Ex: $1001.01_2 = [(1) \times 2^3] + [(0) \times 2^2] + [(0) \times 2^1] + [(1) \times 2^0] + [(0) \times 2^{-1}] + [(1) \times 2^{-2}]$
 $1001.01_2 = [1 \times 8] + [0 \times 4] + [0 \times 2] + [1 \times 1] + [0 \times 0.5] + [1 \times 0.25]$
 $1001.01_2 = 9.25_{10}$

Decimal Number system:

The decimal system has ten symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. In other words, it has a base of 10.

Octal Number System:

Digital systems operate only on binary numbers. Since binary numbers are often very long, two shorthand notations, octal and hexadecimal, are used.

Hexa Decimal Number System:

The hexadecimal numbering system has a base of 16. There are 16 symbols. The decimal digits 0 to 9 are used as the first ten digits, and the letters A through F represent the remaining six digits.

Decimal | Binary | Octal | Hexadecimal

Decimal	Binary	Octal	Hexadecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Number Base conversions:

The human beings use decimal number system while computer uses binary number system. Therefore it is necessary to convert

i) Binary to octal number conversion: Group binary bits into sets of 3 from right to left.

Example: 001 010 011 000 100 101 110 111, = 1 2 3 0 4 5 6 7[^]

ii) Binary to hexadecimal number conversion: Group binary bits into sets of 4 from right to left.

Example: 0001 0010 0100 1000 1001 1010 1101 1111, = 1 2 5 8 9 A D F^{•†}

iii) Octal to binary Conversion: Replace each octal digit with its 3-bit binary equivalent.

Example: 653[^] = 110 101 011,

iv) Hexadecimal to binary conversion: Replace each hex digit with 4-bit binary.

Example: 9B4^{•†} = 1001 1011 0100,

v) Octal to Decimal conversion:

Ex: convert 4057.06[^] to decimal:

$$= 4 \times 8^3 + 0 \times 8^2 + 5 \times 8^1 + 7 \times 8^0 + 0 \times 8^{-1} + 6 \times 8^{-2}$$

$$= 2048 + 0 + 40 + 7 + 0 + 0.09375$$

$$= 2095.09375^{•€}$$

vi) Decimal to Octal Conversion:

Ex: convert 378.93_{10} to octal:Integer part (378_{10}): Successive division by 8:

$$378 / 8 = 47 \text{ remainder } 2$$

$$47 / 8 = 5 \text{ remainder } 7$$

$$5 / 8 = 0 \text{ remainder } 5 \rightarrow \text{Reading upwards: } 572_8$$

Fractional part (0.93_{10}): Successive multiplication by 8:

$$0.93 \times 8 = 7.44 \text{ (int 7)}$$

$$0.44 \times 8 = 3.52 \text{ (int 3)}$$

$$0.52 \times 8 = 4.16 \text{ (int 4)}$$

$$0.16 \times 8 = 1.28 \text{ (int 1)} \rightarrow \text{Reading downwards: } 0.7341_{10}$$

$$\text{Result: } 378.93_{10} = 572.7341_8$$

vii) Hexadecimal to Decimal Conversion:

Ex: $5C7_{16}$ to decimal:

$$= (5 \times 16^2) + (C \times 16^1) + (7 \times 16^0)$$

$$= (5 \times 256) + (12 \times 16) + (7 \times 1) = 1280 + 192 + 7 = 1479_{10}$$

viii) Decimal to Hexadecimal Conversion:

Ex: 2598.675_{10} to hexadecimal:Integer part (2598_{10}): Successive division by 16:

$$2598 / 16 = 162 \text{ remainder } 6$$

$$162 / 16 = 10 \text{ remainder } 2 \text{ (10 = A)}$$

$$10 / 16 = 0 \text{ remainder } 10 \text{ (A)} \rightarrow \text{Reading upwards: } A26_{16}$$

Fractional part: $0.675 \times 16 = 10.8$ (int 10 = A)

$0.800 \times 16 = 12.8$ (int 12 = C)

$0.800 \times 16 = 12.8$ (int 12 = C)

$0.800 \times 16 = 12.8$ (int 12 = C) $\rightarrow 0.ACCC \cdot \uparrow$

Result: $2598.675 \cdot \epsilon = A26.ACCC \cdot \uparrow$

ix) Octal to hexadecimal conversion:

The simplest way is to first convert the given octal no. to binary & then the binary no. to hexadecimal.

Ex: $756.603^{\wedge}$

Octal: 7 5 6 . 6 0 3

Binary: 111 101 110 . 110 000 011

Hex Regroup (4s): 0001 1110 1110 . 1100 0001 1000

Hex Value: 1 E E . C 1 8 $\cdot \uparrow \rightarrow 1EE.C18 \cdot \uparrow$

x) Hexadecimal to octal conversion:

First convert the given hexadecimal no. to binary & then the binary no. to octal.

Ex: $B9F.AE \cdot \uparrow$

Hex: B 9 F . A E

Binary: 1011 1001 1111 . 1010 1110

Octal Regroup (3s): 101 110 011 111 . 101 011 100

Octal Value: 5 6 3 7 . 5 3 4 $\wedge = 5637.534^{\wedge}$

Complements:

In digital computers to simplify the subtraction operation & for logical manipulation complements are used. There are two types of

i) The radix complement or r's complement (2's, 10's complement)

ii) The diminished radix complement or (r-1)'s complement (1's, 9's complement)

Representation of signed numbers binary arithmetic in computers:

- Two ways of representing signed numbers:

1. Sign Magnitude form
2. Complemented form

- Two complemented forms:

1. 1's complement form
2. 2's complement form

Advantage of performing subtraction by the complement method is reduction in the hardware (instead of separate adder & subtractor). i.e., subtraction is also performed by adders only.

Instead of subtracting one number from another, the complement of the subtrahend is added to minuend.

In sign magnitude form, an additional bit called the sign bit is placed in front of the number. If the sign bit is 0, the number is positive.

Example: 0101001 = +41 magnitude (sign bit 0)

1101001 = -41 magnitude (sign bit 1)

Representation of signed numbers using 2's or 1's complement method:

If the number is +ve, magnitude is represented in true binary form with sign bit 0.

If the number is -ve, magnitude is represented in 2's or 1's complement form with sign bit 1.

Given number	Sign-Mag form	2's comp form	1's comp form
01101	+13	+13	+13
010111	+23	+23	+23
10111	-7	-7	-8
1101010	-42	-22	-21

Special case in 2's complement representation:

Whenever a signed number has a 1 in the sign bit & all 0's for the magnitude bit, it represents a negative number. Ex: 1000 = -8, and 10000 = -16.

Characteristics of 2's complement numbers:

1. There is one unique zero (0000 = +0; no -0 representation).
2. 2's complement of 0 is 0.
3. The leftmost bit cannot be used to express a quantity; it is a sign bit (0 for +ve, 1 for -ve).
4. For an n -bit word which includes the sign bit, there are $(2 \cdot \{^1 - 1\})$ positive numbers.
5. Significant information is contained in the 1's of the +ve numbers & 0's of the -ve numbers.
6. A negative number may be converted into a positive number by finding its 2's complement.

Signed Binary Numbers Table (4-bit system):

Decimal | 2's Comp Form | 1's Comp Form | Sign-Mag Form

Decimal	2's Comp Form	1's Comp Form	Sign-Mag Form
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	--	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111
-8	1000	--	--

Methods of obtaining 2's complement of a number:

In 3 ways:

1. **Method I:** By obtaining the 1's complement of the given number (by inverting all 0's to 1's & 1's to 0's) and then adding 1.
2. **Method II:** By subtracting the given n-bit number N from 2^n .
3. **Method III:** Starting at the LSB, copying down each bit up to & including the first 1 bit encountered, and complementing the remaining bits.

Ex: Express -45 in 8-bit 2's complement form:

+45 in 8-bit form is: 00101101

Method I:

1's complement of 00101101 = 11010010

Add 1: + 1

2's complement form = 11010011

Method II: Subtract from $2^n = 100000000 - 00101101 = 11010011$

Method III:

Original number: 00101101

Copy up to first 1: ...1

Complement remaining: 1101001... -> 11010011

Example: Express -73.75 in 12-bit 2's complement form:

Method I:

+73.75 in binary: 01001001.1100

1's complement: 10110110.0011

Add 1 to LSB: + 1

Result: 10110110.0100 is 2's complement form

2's complement Arithmetic:

- **The 2's complement system is used to represent -ve numbers using modulus arithmetic. The word length of a computer is 16 bits.**
- **Subtraction Procedure: In 2's complement subtraction, add the 2's complement of the subtrahend to the minuend.**
 - If there is an end-carry, DISCARD (ignore) it. The result is POSITIVE and in true binary form.
 - If there is NO end-carry, the result is NEGATIVE and in 2's complement form. Take its 2's complement to obtain the true magnitude.

Example: Subtract 14 from 46 using 8-bit 2's complement arithmetic:

+46 = 00101110

+14 = 00001110 -> -14 in 2's comp = 11110010

Add:

00101110 (+46)

+ 11110010 (-14)

(1)00100000 -> End carry (1) is ignored.

MSB is 0, so result is +ve: +00100000 = +32•€.

Example: Add -75 to +26 using 8-bit 2's complement arithmetic:

+75 = 01001011 -> -75 in 2's comp = 10110101

+26 = 00011010

Add:

00011010 (+26)

+ 10110101 (-75)

11001111 -> No carry generated.

MSB is 1, so result is negative and in 2's complement form.

Taking 2's complement of 11001111 = 00110001 = 49•€. Result = -49•€.

1's complement of a number:

- **Obtained by simply inverting each bit (0 becomes 1, 1 becomes 0).**
- **Disadvantage: 1's complement has two representations of zero: +0 (00000000) and -0 (11111111).**

1's complement arithmetic:

In 1's complement subtraction, add the 1's complement of the subtrahend to the minuend.

- **If there is an end-around carry, ADD it to the LSB (end-around carry). The result is positive and in true binary.**
- **If there is NO carry, the result is negative and in 1's complement form. Take 1's complement to find true magnitude.**

Example: Subtract 14 from 25 using 8-bit 1's complement:

25 = 00011001

14 = 00001110 -> 1's comp = 11110001

Add: 00011001 + 11110001 = (1)00001010

End around carry: 00001010 + 1 = 00001011 = +11•€.

Binary codes:

Binary codes represent data in binary systems with modifications from natural binary. Classification:

1. **Weighted Binary codes:** Each position has a specific weight. Examples: 8421 (BCD), 2421, 5211, 84-2-1, Bi-Quinary 50
2. **Non-Weighted Codes:** Positions do not have assigned mathematical weights. Examples: Excess-3 code, Gray code.

Reflective Code:

A code is said to be reflective (self-complementing) when the code for 9 is the 1's complement of the code for 0, code for 8 is complement of 1, and so on. Codes 2421, 5211, and Excess-3 are reflective, whereas standard 8421 code is not.

Sequential Codes:

A code is said to be sequential when two subsequent code words, seen as numbers in binary representation, differ by one. This g

Excess-3 Code (XS-3):

Excess-3 is a non-weighted BCD code used to express decimal numbers. The code derives its name from the

Gray Code (Unit Distance Code):

The Gray code belongs to a class of codes called minimum change codes, in which only one bit in the code changes when moving

The Gray code is non-weighted and reflective. Because adjacent numbers differ by only one bit, it prevents race conditions and gl

Decimal	Binary (8421)	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Binary to Gray Conversion Algorithm:

- Gray Code MSB is equal to Binary MSB: $G^{\text{TM}} = B^{\text{TM}}$
- Subsequent bits: $G_b = B_b \oplus B_{b+1}$ (XOR of adjacent binary bits).

8421 BCD Code (Natural BCD):

Each decimal digit 0 through 9 is coded by a 4-bit binary group (8-4-2-1 weights).

Invalid/Illegal states in BCD: 1010, 1011, 1100, 1101, 1110, 1111 (digits 10 through 15).

BCD Addition Rules:

1. Add the two BCD numbers using ordinary 4-bit binary addition.
2. If the 4-bit sum is ≤ 9 and no carry is generated, the sum is in valid BCD.
3. If the 4-bit sum > 9 or a carry out is generated, the sum is invalid. Add correction factor 0110 (6) to the sum and propagate the carry.

Example: 25 + 13 in BCD:

25 = 0010 0101
+ 13 = 0001 0011

38 = 0011 1000 (Sum ≤ 9 in both decades, no carry $\rightarrow$ Valid BCD: 38).

Example: $679.6 + 536.8$ in BCD:

$679.6 = 0110\ 0111\ 1001 . 0110$

$+ 536.8 = 0101\ 0011\ 0010 . 1000$

Binary: $1011\ 1010\ 1011 . 1110$ (all illegal codes > 9)

Correction: $+0110$ to each group with carry propagation:

Result: $0001\ 0010\ 0001\ 0110 . 0100 = 1216.4$ BCD.

BCD Subtraction:

Performed by subtracting 4-bit groups. If a borrow occurs from the next higher decade, subtract 0110 (6) from that group to restore

Example (a): $38 - 15 = 23$ (no borrow)

Example (b): $206.7 - 147.8 = 58.9$ (borrows occur, corrected by -0110).

BCD Subtraction using 9's and 10's Complement:

Form the 9's complement of the subtrahend, encode into BCD, and add to minuend.

Example: $305.5 - 168.8 = 136.7$

Excess-3 Addition Rules:

1. Add the two Excess-3 numbers in 4-bit groups.
2. If a carry is generated from the 4-bit addition, ADD 0011 (3) to that group.
3. If NO carry is generated, SUBTRACT 0011 (3) from that group (i.e. add 1101).

Example: Excess-3 Addition of $37 + 28 = 65$:

37 in XS-3 = 0110 1010

+ 28 in XS-3 = 0101 1011

Sum: 1011 (1)0101 (carry out of lower group)

Correction: -0011 +0011

Result: 1001 1000 = 65 in XS-3.

Excess-3 Subtraction:

Subtract 4-bit groups. If no borrow occurs, add 0011. If borrow occurs, subtract 0011.

Example: $267 - 175 = 92$.

Excess-3 subtraction using 9's and 10's complement:

Example: $687 - 348 = 339$

9's complement of 348 = 651

Adding in XS-3 code and applying end-around carry yields corrected difference 0110 0110 1100 = 339 in decimal.

The Gray code is a reflective code. An n-bit Gray code can be generated by reflecting the (n-1)-bit code about an axis, prefixing 0

Table of 1-bit, 2-bit, 3-bit, and 4-bit Gray code reflections.

Error-Detecting Codes: When binary data is transmitted through communication channels, noise can flip bits (0 to 1 or 1 to 0).

Parity Bit: A parity bit is appended to the data word.

- **Even Parity: Total number of 1s in the transmitted word (including parity bit) is EVEN.**
- **Odd Parity: Total number of 1s in the transmitted word (including parity bit) is ODD.**

Parity Checking Circuits: An XOR tree checks parity at the receiver. A single parity bit detects single-bit errors, but cannot detect t

Checksums: Two-dimensional summation of data blocks.

Block Parity (Horizontal and Vertical Parity): Matrix arrangement of data where both rows and columns have parity bits, allowing d

Error-Correcting Codes: To detect and correct an error, the minimum Hamming distance must be at least 3.

Hamming Code: Parity check bits P_k are placed at power-of-2 positions: 1, 2, 4, 8, ... ($2^{(k-1)}$).

Parity equations check specific overlapping bit subsets.

7-Bit Hamming Code format: P1, P2, D3, P4, D5, D6, D7

Parity bit coverage:

- **P1 checks bits 1, 3, 5, 7**
- **P2 checks bits 2, 3, 6, 7**
- **P4 checks bits 4, 5, 6, 7**

Syndrome Word $S = S_3 S_2 S_1$ directly yields the binary address of the erroneous bit.

Example: Encode data 1101 into 7-bit even parity Hamming Code -> 1010101.

Example: Syndrome decoding of received word 1001001 -> Syndrome = 010 (bit 2 is in error).

12-Bit and 15-Bit Hamming Code formats.

Alphanumeric Codes: ASCII (7-bit / 8-bit) and EBCDIC (8-bit) character encoding systems.

Digital Logic Gates:

Boolean functions are expressed in terms of logic operations implemented by hardware gates:

- **AND Gate:** $F = x \cdot y$ (Outputs 1 only when all inputs are 1)
- **OR Gate:** $F = x + y$ (Outputs 1 when at least one input is 1)
- **NOT Gate (Inverter):** $F = x'$ (Inverts logic state)
- **Buffer:** $F = x$ (Current amplification and signal restoration)
- **NAND Gate:** $F = (x \cdot y)'$ (Inverted AND)
- **NOR Gate:** $F = (x + y)'$ (Inverted OR)
- **XOR Gate:** $F = x \oplus y = x'y + xy'$ (Odd parity detector)
- **XNOR Gate:** $F = (x \oplus y)' = xy + x'y'$ (Equivalence gate)

Properties of XOR Gates:

- $x \oplus 0 = x$; $x \oplus 1 = x'$; $x \oplus x = 0$; $x \oplus x' = 1$
- Commutative: $x \oplus y = y \oplus x$
- Associative: $(x \oplus y) \oplus z = x \oplus (y \oplus z)$

Universal Logic Gates (NAND & NOR):

NAND and NOR can independently synthesize all basic logic operations (NOT, AND, OR):

- NOT using NAND: Connect inputs together: $(x \cdot x)' = x'$
- AND using NAND: NAND gate followed by NAND inverter: $((x \cdot y)')' = xy$
- OR using NAND: Invert inputs before NAND: $(x' \cdot y')' = x + y$ (by De Morgan)

Boolean Algebra & Switching Algebra:

Developed by George Boole (1854) and adapted to switching circuits by Claude Shannon (1938). Huntington Postulates (1904).

Axioms:

- **AND:** $0 \cdot 0 = 0$, $0 \cdot 1 = 0$, $1 \cdot 0 = 0$, $1 \cdot 1 = 1$
- **OR:** $0 + 0 = 0$, $0 + 1 = 1$, $1 + 0 = 1$, $1 + 1 = 1$
- **NOT:** $0' = 1$, $1' = 0$
- **Null laws:** $A \cdot 0 = 0$, $A + 1 = 1$
- **Identity laws:** $A \cdot 1 = A$, $A + 0 = A$
- **Idempotent laws:** $A \cdot A = A$, $A + A = A$
- **Closure:** System is closed under binary operations $+$ and $\cdot$.

Basic Laws & Identities:

- **Commutative:** $A + B = B + A$; $A \cdot B = B \cdot A$
- **Associative:** $(A + B) + C = A + (B + C)$; $(AB)C = A(BC)$
- **Distributive:** $A(B + C) = AB + AC$; $A + BC = (A + B)(A + C)$
- **Complementarity:** $A + A' = 1$; $A \cdot A' = 0$
- **Involution:** $(A')' = A$
- **De Morgan's Laws:** $(A + B)' = A'B'$; $(AB)' = A' + B'$

De Morgan's Theorems:

1. $(A + B)' = A' \cdot B'$

2. $(A \cdot B)' = A' + B'$

Generalized De Morgan: $(A + B + \dots + Z)' = A'B' \dots Z'$

Absorption Laws: $A + AB = A$; $A(A + B) = A$

Consensus Theorem:

- Theorem 1: $AB + A'C + BC = AB + A'C$ (BC is redundant consensus term)
- Proof: $AB + A'C + BC = AB + A'C + BC(A + A') = AB(1 + C) + A'C(1 + B) = AB + A'C$.
- Theorem 2 (Dual): $(A + B)(A' + C)(B + C) = (A + B)(A' + C)$

Principle of Duality:

Any true Boolean algebraic relation remains true if operator + is replaced by $\cdot$, $\cdot$ is replaced by +, and identity elements 0 and 1 are interchanged.

Boolean Function representation: $F(\text{vars}) = \text{expression}$.

Example: $F1 = x + y'z$.

Truth Table representation of Boolean functions:

For n variables, there are 2^n rows.

Truth table for $F1 = x + y'z$ with inputs x, y, z .

Logic gate implementation of $F1$ using AND, OR, NOT gates.

Truth tables provide unique representations of Boolean functions, but algebraic expressions are NOT unique.

Algebraic Minimization reduces gate count, chip area, power consumption, and propagation delay.

Example: Simplify $F = x'yz + x'yz' + xz = x'y(z + z') + xz = x'y + xz$.

Complement of a Function: Invert all literals and swap operators (or negate truth table outputs).

Example: $F = xy'z' + x'yz \rightarrow F' = (x' + y + z)(x + y' + z')$.

Canonical and Standard Forms:

- **Literal:** A variable or its complement (A or A').
- **Product term:** Literals connected by AND (e.g. ABC).
- **Sum term:** Literals connected by OR (e.g. A + B + C).
- **Minterm (m_j):** A product term containing every variable once.
- **Maxterm (M_j):** A sum term containing every variable once.

Minterms (m0 to m7) and Maxterms (M0 to M7) for 3-variable system (x, y, z):

x y z			Minterm (m _j)	Maxterm (M _j)
-----+-----+-----				
0	0	0	$x'y'z' = m_0$	$x + y + z = M_0$
0	0	1	$x'y'z = m_1$	$x + y + z' = M_1$
0	1	0	$x'yz' = m_2$	$x + y' + z = M_2$
0	1	1	$x'yz = m_3$	$x + y' + z' = M_3$
1	0	0	$xy'z' = m_4$	$x' + y + z = M_4$
1	0	1	$xy'z = m_5$	$x' + y + z' = M_5$
1	1	0	$xyz' = m_6$	$x' + y' + z = M_6$
1	1	1	$xyz = m_7$	$x' + y' + z' = M_7$

Every Boolean function has two unique canonical forms:

1. Canonical Sum-Of-Products (SOP): Sum of minterms for which $F = 1$.

2. Canonical Product-Of-Sums (POS): Product of maxterms for which $F = 0$.

Example: $f_1(a, b, c) = m_1 + m_2 + m_4 + m_6 = a'b'c + a'bc' + ab'c' + abc'$.

POS form: $f_1 = M_0 \cdot M_3 \cdot M_5 \cdot M_7 = (a+b+c)(a+b'+c')(a'+b+c')(a'+b'+c')$.

Property: $m_j = (M_j)'$.

Shorthand Notations:

- **Sum of Minterms:** $f_1(a,b,c) = \sum m(1, 2, 4, 6)$
- **Product of Maxterms:** $f_1(a,b,c) = \prod M(0, 3, 5, 7)$
- **Conversion rule:** Replace $\sum$ with $\prod$ and list missing decimal indices.

Standard Forms:

- **SOP:** Sum of product terms (e.g. $F_1 = y' + xy + x'yz'$).
- **POS:** Product of sum terms (e.g. $F_2 = x(y' + z)(x' + y + z')$).

Conversion of standard SOP to canonical SOP:

Multiply terms by $(x + x')$ for each missing variable.

Example 1: $F = A + B'C = \sum m(1, 4, 5, 6, 7)$.

Example 2: $F = xy + x'z = \sum m(0, 2, 4, 5)$.

UNIT - II: MINIMIZATION TECHNIQUES

Two-variable K-Map ($2^2 = 4$ cells):

Each square corresponds to a unique minterm: $m_0 (A'B')$, $m_1 (A'B)$, $m_2 (AB')$, $m_3 (AB)$.

Truth table mapping to 2x2 grid.

Adjacency: Horizontal and vertical neighbors differ by exactly one bit.

Mapping examples: $F = \sum m(0, 2, 3)$ and $F = \sum m(1, 2)$.

Minimization of SOP expressions: adjacent 1s form pairs (2-squares) to eliminate 1 literal.

Pair (2-square) eliminates 1 literal; Quad (4-square) eliminates 2 literals.

Example: Reduce $F = \sum m(0, 1, 3) \rightarrow f = A' + B$. Logic diagram implementation.

Minimization in POS form: Group adjacent 0s into pairs and quads.

Each sum term reads literals that remain constant across the group (0 -> uncomplemented, 1 -> complemented).

Three-variable K-map ($2^3 = 8$ cells):

Rows: A (0, 1); Columns: BC (00, 01, 11, 10 in Gray code order).

Adjacency via Gray code ensures physically neighboring squares differ by only 1 literal.

Mapping examples in 3-variable map:

- **SOP:** $f = \sum m(1, 2, 5, 6, 7)$
- **POS:** $f = \prod M(0, 3, 5, 6, 7)$

General procedure for K-map reduction:

1. Plot 1s (or 0s for POS).
2. Identify isolated 1s (cannot be combined).
3. Identify 1s with only 1 neighbor and form 2-squares.
4. Form largest possible quads (4-squares) and octets (8-squares).
5. Avoid redundant groups.

Examples of reading minimized terms from 2-squares, 4-squares, and wrap-around edge adjacencies.

Example: Minimize $F = \sum m(0, 2, 3, 4, 5, 6) \rightarrow F_{\min} = C' + AB' + A'B$.
Realization using AOI logic and all-NAND logic gates.

Four-variable K-map (2t = 16 cells):
Rows: AB (00, 01, 11, 10); Columns: CD (00, 01, 11, 10).
Wraps horizontally and vertically (corner cells m0, m2, m8, m10 form a 4-square quad).

Five-variable K-map (2u = 32 cells):

Composed of two 16-cell sub-cubes: Map 1 for A=0, Map 2 for A=1.

Adjacency exists between identical row-column positions across Map 1 and Map 2.

Example: Minimize $F = \sum m(0, 1, 4, 5, 6, 13, 14, 15, 22, 24, 25, 28, 29, 30, 31)$.

Grouping 8-squares and 4-squares yields $F_{\min} = A'BCD' + B'C'E' + AB'D' + C'D$.

Six-variable K-map ($2^6 = 64$ cells, four 16-cell sub-cubes).
Don't Care conditions (d): Represent unused input states (e.g. in BCD or XS-3). Can be treated as 1s to enlarge groups or ignored.

Definitions:

- **Implicant:** Any single minterm or group of minterms.
- **Prime Implicant (PI):** A maximal group that cannot be merged into a larger group.
- **Essential Prime Implicant (EPI):** A PI covering at least one 1 not covered by any other PI.
- **Redundant Prime Implicant (RPI):** A PI whose 1s are all covered by EPIs.

Selective Prime Implicants (SPI): PIs chosen to cover remaining minterms.
False PIs (FPI) and Essential False PIs (EFPI) in POS minimization.

Quine-McCluskey (Tabular) Minimization Method:

Systematic algorithmic method suitable for computer automation and functions with > 6 variables.

Partitions minterms into groups by index (number of 1s in binary representation).

Example: Find all Prime Implicants for $F = \sum m(0, 1, 6, 7, 8, 9, 13, 14, 15)$.
Grouping into index sets, comparing adjacent groups, and marking combined terms.

Generating 2-variable and 4-variable implicants with dash (-) notation for eliminated literals.

Forming Prime Implicant Table (Rows = PIs, Columns = Minterms). Checkmarks for minterms covered by each PI.

Identifying Essential Prime Implicants (columns with a single checkmark).
Eliminating covered columns and applying row dominance to find minimal cover.

Obtaining final minimal SOP equation: $F = BD + AC' + A'C + C'D$.

Comparison between K-map and Tabular methods.

UNIT - III: COMBINATIONAL CIRCUITS

Combinational Logic definition: Output depends solely on present input values (no memory/feedback).

Design Procedure (5 steps):

- 1. Problem statement**
- 2. Determine inputs & outputs**
- 3. Assign letter symbols**
- 4. Derive truth table**
- 5. Obtain simplified Boolean functions & draw logic schematic.**

Binary Arithmetic operations: $0+0=0$, $0+1=1$, $1+0=1$, $1+1=10$.

Half Adder: Adds two 1-bit inputs A and B.

- **Sum:** $S = A \oplus B = A'B + AB'$

- **Carry:** $C = AB$

Truth table, block diagram, and logic diagram using XOR and AND gates.

Half Adder realization using only 2-input NAND gates (5 gates).

Half Adder realization using only 2-input NOR gates (5 gates).

The Full Adder: Adds three binary bits (A, B, and incoming Carry C_{in}).

Full Adder Truth Table (8 rows).

Equations:

- **Sum: $S = A \oplus B \oplus C_{in}$**
- **Carry Out: $C_{out} = AB + C_{in}(A \oplus B) = AB + BC_{in} + AC_{in}$**

Implementation using 2 Half Adders and 1 OR gate.

Full Adder circuit realization using 9 two-input NAND gates.

Propagation delay comparison between 2-level AOI and cascaded Half Adders.

Full Adder realization using 9 NOR gates.

Subtractors: Subtraction by complement addition.

Half Subtractor: Subtracts 1-bit B from A.

- **Difference:** $d = A \oplus B$
- **Borrow Out:** $b = A'B$

Half Subtractor logic diagram using XOR and AND with inverted input.
NAND logic realization (5 gates) and NOR logic realization (5 gates).

Full Subtractor: Subtracts B and borrow-in b_i from minuend A.

Equations:

- **Difference:** $d = A \oplus B \oplus b_i$
- **Borrow Out:** $b = A'B + b_i(A \oplus B)' = A'B + A'b_i + Bb_i$

Full Subtractor realization using two half subtractors and an OR gate.
Universal NAND logic and NOR logic schematic implementations.

4-Bit Binary Parallel Adder: Cascade of 4 Full Adders (FA0 to FA3).

Carry out of each stage connects to carry in of the next higher significant stage.

MSI building block: 74LS83 / 74LS283 4-bit binary adder.

Propagation delay in ripple carry adders: Total delay = $n \times t_{\text{carry}}$.

4-Bit Parallel Subtractor using 2's complement.

Binary Adder-Subtractor: Controlled by Mode input M (M=0: Addition; M=1: Subtraction using XOR inverters and C0=1).

Carry Look-Ahead (CLA) Adder:

Eliminates ripple carry propagation delay by computing all carries in parallel.

- **Carry Generate:** $G_b = A_b \cdot B_b$
- **Carry Propagate:** $P_b = A_b \oplus B_b$

Recursive Carry Equations:

$$\bullet C_0 = G_0 + P_0 C_{-1}$$

$$\bullet C_1 = G_1 + P_1 G_0 + P_1 P_0 C_{-1}$$

$$\bullet C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{-1}$$

$$\bullet C_n = G_n + P_n G_{n-1} + P_n P_{n-1} G_{n-2} + \dots + P_n P_{n-1} P_{n-2} \dots P_1 P_0 C_{-1}$$

All carries are generated in 2 gate delays regardless of word size.

Logic schematic of 4-bit CLA Carry Generator.

2's complement addition and subtraction circuit using parallel adders with register gating.

Serial Adder: Processes 1 bit per clock cycle using two shift registers (A and B), a single Full Adder, and a D

Hardware-efficient architecture for low-cost systems.

Comparison: Parallel Adder (fast, higher hardware cost) vs Serial Adder (slow, minimal hardware).

BCD Adder principles: Addition of decimal digits in 8421 code with +0110 (6) correction.

Correction detection condition: $X = S_{10} + S f(S_9 + S_8) = 1$ when sum > 9 or carry out

Two-stage adder structure: 4-bit binary adder followed by correction adder.

Cascading multiple BCD adder stages for multi-digit decimal addition.

Excess-3 Adder circuit implementation using dual 4-bit binary adders.

Excess-3 Subtractor circuit implementation.

Binary Multiplication principles: Shift-and-add partial product accumulation.

Hardware implementation of 4-bit binary multiplier using Shift Registers (X, B, A) and an 8-bit parallel adder with control sequence

Code Converters: Combinational mapping between distinct digital codes.

4-bit Binary to Gray Code converter design:

- $G_f = B_f$
- $G_3 = B_3 \oplus B_2$
- $G_2 = B_2 \oplus B_1$
- $G_1 = B_1 \oplus B_0$

4-bit Gray to Binary Code converter design:

- $B_f = G_f$
- $B_3 = G_f \oplus G_3$
- $B_2 = G_f \oplus G_2 \oplus G_3$
- $B_1 = G_f \oplus G_1 \oplus G_2 \oplus G_3$

Logic diagram using cascaded XOR gates.

4-bit BCD to XS-3 Code Converter design using K-maps:

- $X_f = B_f + B, B \cdot + B, B \epsilon$
- $X, = B, 'B \cdot + B, 'B \epsilon + B, B \cdot 'B \epsilon '$
- $X \cdot = B \cdot \text{ "TM } B \epsilon$
- $X \epsilon = B \epsilon '$

BCD to Gray code converter design.

SOP detector circuit for numbers 5 through 12 in 4-bit Gray code.

Detector circuit for decimal 0, 2, 4, 6, 8 in 5211 BCD code.

Combinational circuit producing 2's complement of 4-bit number.

Seven-segment display decoder principles.

Magnitude Comparators: Compare two binary numbers A and B to evaluate $A > B$, $A < B$, $A = B$.

1-bit Magnitude Comparator equations:

- $A > B: G = AB'$

- $A < B: L = A'B$

- $A = B: E = A \oplus B = AB + A'B'$

2-bit Magnitude Comparator equations and logic schematic.

4 - Bit Magnitude Comparator ($A \neq B$ vs $A > B$):

Iterative comparison starting from MSB downwards.

Logic diagram with cascaded XNOR and AND-OR stages.

74LS85 4-bit Magnitude Comparator IC pinout and expansion cascading for 8-bit and 16-bit comparisons.

Encoders: Convert 2ⁿ active input lines into an n-bit binary output code.
Octal to Binary Encoder (8-to-3).

Decimal to BCD Priority Encoder (74LS147).

Tristate Bus System: Three output states (Logic 0, Logic 1, High-Impedance Z). Output Enable (OE) control for multiplexing on shared bus.

UNIT - IV: SEQUENTIAL CIRCUITS

Sequential circuits contain memory elements and feedback loops (Next State = $f(\text{Present State, Inputs})$).

The Basic Latch: Cross-coupled NOR gates or NAND gates.

Gated SR Latch and Gated D Latch: Control/Enable input allows state changes only when active.

Gated D Latch circuit, characteristic table, and timing waveforms.

Dynamic Timing parameters:

- **Setup Time (t_{su}):** Minimum time data must remain stable BEFORE clock edge.
- **Hold Time (t_h):** Minimum time data must remain stable AFTER clock edge.

Metastability occurs if setup or hold times are violated.

Flip-Flops are edge-triggered storage elements.

Master-Slave D Flip-Flop: Two cascaded latches clocked on opposite phases (Master active during High, Slave transfers during L

Master-Slave JK Flip-Flop with asynchronous Preset and Clear inputs.

Characteristic Equation: $Q(t+1) = JQ' + K'Q$.

T (Toggle) Flip-Flop: Formed by tying J and K together ($Q(t+1) = T \oplus Q$).

Excitation Tables for D, JK, SR, and T Flip-Flops:

Transition	S	R	J	K	D	T
0 -> 0	0	X	0	X	0	0
0 -> 1	1	0	1	X	1	1
1 -> 0	0	1	X	1	0	1
1 -> 1	X	0	X	0	1	0

Conversion of Flip-Flops methodology:

1. Draw transition table with required next states.
2. Fill in excitation values for available flip-flop.
3. Use K-maps to solve for available inputs.

Example: Realize D-FF using JK-FF ($J=D$, $K=D'$).

Example: Implement JK-FF using D-FF ($D = JQ' + K'Q$).

8-Step Design Procedure for Synchronous Sequential Circuits (State Diagram, State Table, State Assignment, Excitation, K-maps)
Shift Registers: Cascade of flip-flops sharing common clock for serial/parallel data manipulation.

Parallel-Access Shift Register with Mode control (Shift vs Parallel Load).

Introduction to Counters: Register circuits progressing through predetermined sequence of binary states on clock transitions.

2-Bit Binary Counter State Table (00 -> 01 -> 10 -> 11 -> 00).

Applications of counters: Real-time clocks, frequency dividers, sequence generators, CPU Program Counters (PC).

3-Bit Asynchronous (Ripple) Up-Counter using T flip-flops.

Clock is applied to LSB; each subsequent stage is clocked by inverted output Q' of previous stage.

3-Bit Asynchronous Down-Counter circuit and timing waveforms.

Classification of Shift Registers: SISO, SIPO, PISO, PIPO, and Bidirectional.

Buffer Registers: 4-bit data latching with Tri-state output drivers.

Block diagrams and operation of Serial-In Serial-Out, Serial-In Parallel-Out, Parallel-In Serial-Out, Parallel-In Parallel-Out registers

Detailed circuit schematic and timing operation of 4-bit SISO and SIPO shift registers.

Detailed circuit schematic and timing operation of 4-bit PISO and PIPO shift registers.

Bidirectional Shift Register: Steered by Mode control signal (Right/Left) via AND-OR multiplexing gates between adjacent flip-flops

4-Bit Universal Shift Register implementation using 4x1 Multiplexers and D flip-flops.
Provides Parallel Load, Shift Right, Shift Left, and Hold functions.

Mode Control Function Table (S1S0: 00 Hold, 01 Shift Right, 10 Shift Left, 11 Parallel Load).
Counters Classification: Asynchronous (Ripple) vs Synchronous (Parallel), Decade, Ring, Johnson.

Analysis of 2-bit Ripple Up-Counter using negative-edge triggered JK flip-flops.
State progression: 00 -> 01 -> 10 -> 11 -> 00. Propagation delays and ripple effects.

Analysis of 2-bit Ripple Down-Counter: State progression 00 -> 11 -> 10 -> 01 -> 00.
Timing waveforms and clock edge triggering.

2-Bit Bidirectional Up-Down Ripple Counter using Mode control M.

Design of Asynchronous Counters with arbitrary modulus using reset feedback.

Design of Mod-6 Counter using 3 T flip-flops: Counts 000 to 101, resets at 110

Design of Mod-10 (Decade / BCD) Asynchronous Counter using 4 flip-flops.

Counts 0000 to 1001, resets at 1010 (10) via NAND gate feedback ($R = Q_3 \cdot Q_1$).

Synchronous Counters: All flip-flops are triggered simultaneously by the common clock, eliminating ripple delays.
Systematic 5-step design procedure using state transition tables and excitation K-maps.

Complete State and Excitation Table for Synchronous 3-Bit Up-Down Counter using JK Flip-Flops with Mode input M.

K-maps and logic diagram for Synchronous 3-Bit Up-Down Counter.

Design of Synchronous Modulo-6 Gray Code Counter (Sequence: 000 -> 001 -> 011 -> 010 -> 110 -> 111 -> 000).

Excitation K-maps and logic diagram for Mod-6 Gray Code Counter using T flip-flops.

Complete 20-row state transition and excitation table for Synchronous BCD (Decade) Up-Down Counter using T flip-flops.

Excitation equations for BCD Up-Down Counter.

Ring Counter: Shift register with inverted or direct feedback.

4-bit Ring Counter: Single circulating 1 (States: 1000 -> 0100 -> 0010 -> 0001 -> 1000). Modulus = $N = 4$.

Ring Counter timing diagram.

Twisted Ring Counter (Johnson Counter): Inverted output Q' of last flip-flop feeds back to D input of first stage.

Modulus = $2N$ (4 flip-flops yield 8 distinct states: 0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001).

4-bit Johnson Counter circuit using JK flip-flops.
Complete 8-state sequence table, timing diagram, and decoding gates.

UNIT - V: MEMORY DEVICES

Classification of Semiconductor Memories:

- **Read-Only Memory (ROM):** Non-volatile (PROM, EPROM, EEPROM, Flash).
- **Random-Access Memory (RAM):** Read/Write volatile (Static RAM, Dynamic RAM).

Memory Organization: Address Bus (k bits), Data Bus (n bits), Control lines (Re

Random-Access Memory (RAM): Constant access time independent of physical word storage address.

Sequential-Access Memory: Access time depends on position of read/write head (magnetic tape, hard disks).

Static RAM (SRAM):

- Internal 6-transistor cross-coupled latch stores bit.
- Retains data as long as power is applied; fast access, low density, high power consumption.

Dynamic RAM (DRAM):

- Stores bit as charge on microscopic capacitor with 1 transistor (1T-1C cell).
- High storage density, low cost, but requires periodic refresh cycles (every 64ms) due to charge leakage.

Memory Decoding: 2-dimensional coincidence decoding (Row and Column decoders) minimizes decoding gate count.
Binary Memory Cell logic diagram: SR latch with Select, Read, Write, and Data input/output control lines.

Programmable Logic Array (PLA):

Features a Programmable AND array followed by a Programmable OR array.

Implements sum-of-products Boolean expressions.

Output XOR inverters allow true or complemented output programming.

Example: Realizing $F1 = AB' + AC + A'BC'$ and $F2 = (AC + BC)'$.

PLA Programming Table format: Product terms, Input literals (1, 0, -), Output connections (True/Complement).
Design Problem: Implement $F1(A,B,C) = \sum m(0,1,2,4)$ and $F2(A,B,C) = \sum m(0,5,6,7)$

Simplification using K-maps for both true and complement forms:

- **$F1 = (AB + AC + BC)'$ (3 product terms)**
- **$F2 = AB + AC + A'B'C'$**

Total product terms shared: AB, AC, BC, A'B'C'. PLA Programming Table derivation.

